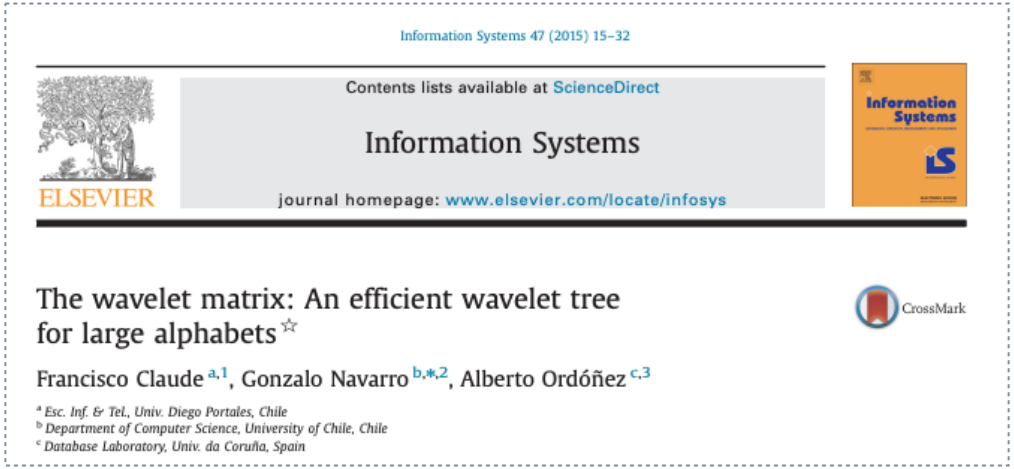


Information Systems 47 (2015) 15–32

Contents lists available at ScienceDirect

Information Systems

journal homepage: www.elsevier.com/locate/infosys

The wavelet matrix: An efficient wavelet tree for large alphabets ☆

Francisco Claude [a,1], Gonzalo Navarro [b,*,2], Alberto Ordóñez [c,3]

[a] Esc. Inf. & Tel., Univ. Diego Portales, Chile
[b] Department of Computer Science, University of Chile, Chile
[c] Database Laboratory, Univ. da Coruña, Spain

An Efficient Wavelet Tree for Large Alphabets

Claude et al. (2015)

Slides created with the assistance of Claude Sonnet (a *different* Claude)

Motivation: Why Do We Need This?

The Problem

Genomic sequences are over large alphabets (e.g., amino acids, k-mers)

Wavelet trees are complex to implement efficiently for large σ

Pointer-based trees waste space & hurt cache performance

We need fast rank/select/access — the backbone of compressed FM-indexes

The Wavelet Matrix Solution

Flat array layout (no pointers) → cache friendly

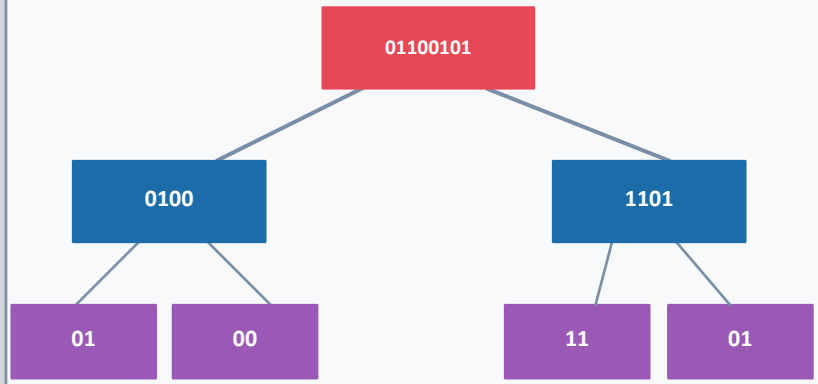
Identical to wavelet tree in bits — same theoretical bounds

Simpler navigation via bit-rank on each level

O(log σ) time for access, rank, select

Wavelet Tree vs. Wavelet Matrix

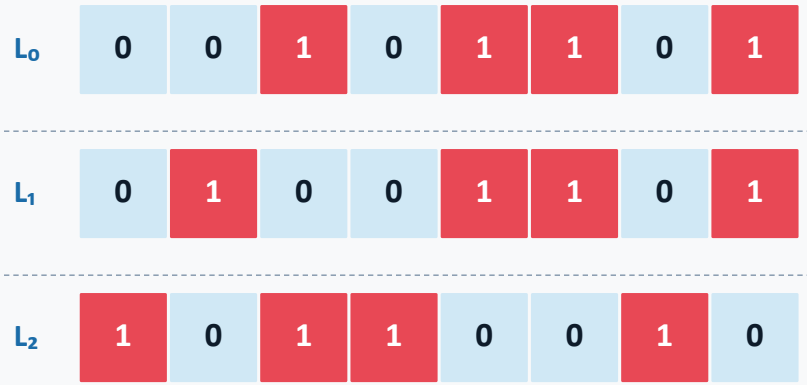
Wavelet Tree



Each level stored in a SEPARATE node
Pointer overhead per node
Navigation follows tree branches

VS

Wavelet Matrix



All levels stored as FLAT BIT ARRAYS
No pointers, no tree nodes
Navigation via bit-rank only

Our Running Example

We will use this sequence throughout:

index:	0	1	2	3	4	5	6	7
S =	**3**	**1**	**4**	**1**	**5**	**9**	**2**	**6**

Alphabet: $\Sigma = \{1,2,3,4,5,6,9\}$ ($\sigma = 9$ distinct values)

Bits per level: $\lceil \log_2 \sigma \rceil = \lceil \log_2 9 \rceil = 4$ levels

Binary representations (4 bits):

`1 → 0001` `2 → 0010` `3 → 0011` `4 → 0100`

`5 → 0101` `6 → 0110` `9 → 1001`

We will answer:

`access(5)`

What is S[5]?

`rank(3, 4)`

How many 3s in S[0..4]?

`select(1, 1)`

Where is the 2nd occurrence of 1?

Construction: Building the Wavelet Matrix

S = [3, 1, 4, 1, 5, 9, 2, 6] (4 levels for σ ≤ 16) ← sequence

Level 0 (bit 3 = MSB)

3	1	4	1	5	9	2	6
0	0	0	0	0	1	0	0

Level 1 (bit 2)

3	1	4	1	5	2	6	9
0	0	1	0	1	0	1	0

Level 2 (bit 1)

3	1	1	2	9	4	5	6
1	0	0	1	0	0	0	1

Level 3 (bit 0 = LSB)

1	1	9	4	5	3	2	6
1	1	1	0	1	1	0	0

z val	← rearranged numbers
z=7	3 1 4 1 5 2 6 9
z=4	3 1 1 2 9 4 5 6
z=5	1 1 9 4 5 3 2 6
done	

The Final Wavelet Matrix Structure

Four bit arrays + four z-values (count of 0s per level)

	index: 0	1	2	3	4	5	6	7	z values →
L_0	0	0	0	0	0	1	0	0	z=7
L_1	0	0	1	0	1	0	1	0	z=4
L_2	1	0	0	1	0	0	0	1	z=5
L_3	1	1	1	0	1	1	0	0	z=3

Each level stores a rank_1 / rank_0 structure for O(1) queries. Space: $n\lceil \log_2 \sigma \rceil$ bits total.

Operation: access(i) — What is S[i]?

Algorithm: access(i)

1. Start with position $p = i$
2. For each level $\ell = 0 \ldots \log_2\sigma - 1$:
   a. bit ← $L_\ell[p]$
   b. if bit == 0:
      $p \leftarrow \text{rank}_0(L_\ell, p)$
   c. else:
      $p \leftarrow z_\ell + \text{rank}_1(L_\ell, p)$
3. Decode final p → symbol

Key Intuition

Each level partitions symbols into 0-half and 1-half. The z value marks the boundary. We follow a position from level to level, tracking which half it falls into — this re-assembles the original symbol bit by bit.

Time: O(log σ) — one bit-rank per level
Space: included in level's rank structure

z_ℓ = number of 0s in level ℓ
= boundary between 0-children and 1-children

access(5) — Step-by-Step Example

Query: access(5) → What is S[5]? (Expected answer: 9)

Result: Bits 1001 = 9 in decimal → **S[5] = 9** ✓

Operation: rank(c, i) — How Many c's up to i?

rank(c, i) = number of occurrences of symbol c in S[0..i]

Algorithm: rank(c, i)

1. $p \leftarrow i$ (position pointer)
2. For each level $\ell = 0 \ldots \log_2\sigma - 1$:
   Let $b \leftarrow$ bit ℓ of c (MSB first)
   if $b == 0$:
   $p \leftarrow \text{rank}_0(L_\ell, p)$
   else:
   $p \leftarrow z_\ell + \text{rank}_1(L_\ell, p)$
3. $\text{rank}(c, i) = p - (\text{start of } c\text{'s range}) + 1$

Intuition

We trace symbol c bit-by-bit from level 0 down. At each level, we follow the same path that c would follow during construction. The final position p tells us where c's occurrences have been mapped.

Symbol Range after Construction

After all levels, identical symbols are grouped contiguously. The range [sp, ep] of symbol c can be computed in O(log σ) time. The answer is:

$$\text{rank}(c, i) = p - sp + 1$$

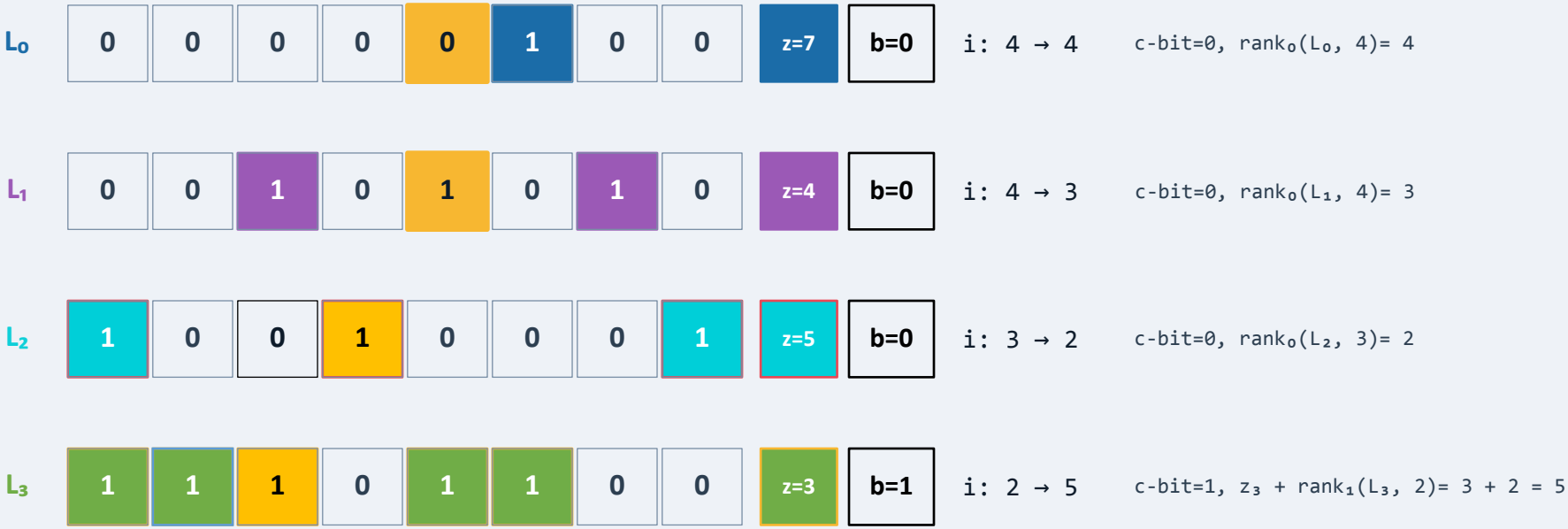
rank(1, 4) — How Many 1s in S[0..4]?

c = 1 = 0001 in binary | i = 4 | Expected: 2 (S = [3,1,4,1,5,9,2,6])

C[a] = starting position for occurrences of symbol ‘a’ in the *last level* of the wavelet matrix

c = 1 = | 0 | 0 | 0 | 1 |

Rank of 1 in S here



Final i = 5, C[1] = 3 → rank(1, 4) = 5 − 3 = **2** ✓ (1 appears at S[1] and S[3])

Operation: select(c, j) — Where is the j-th c?

select(c, j) = position of the j-th occurrence of symbol c in S

Algorithm: select(a, j)

```
1.  j ← C[a] + j  (j-th pos in a's range)
2.  For each level ℓ = log₂σ DOWN to 0:
      Let b ← bit ℓ of a
      if b == 0:
          p ← select₀(Lℓ, j)
      else:
          p ← select₁(Lℓ, j − zℓ)
3.  Return j
```

Key Contrast with rank

rank traverses TOP-DOWN, following bits of c.

select traverses BOTTOM-UP, inverting the mapping layer by layer.

Time: O(log σ) — one bit-select per level

$\text{select}_b(a, k)$ = position of k-th b in a

Intuition: Working Backwards

We know where c lives in the final sorted array. Each $\text{select}_1/_0$ call "unshuffles" one level of the construction, recovering the original position.

select(1, 1) — Where is the last index with 1 rank of 1?

S = [3, 1, 4, 1, 5, 9, 2, 6] should find this index (3)

Step 1: C[1] = 3, so j = 3 + 1

Step 2: Traverse BOTTOM-UP (ℓ = 3 → 0) using select:

c = 0001 (bits at each level: b_3=1, b_2=0, b_1=0, b_0=0, reading LSB→MSB)

L3	1	1	1	0	1	1	0	0	z=3	b=1	j: 4 → 1
L2	1	0	0	1	0	0	0	1	z=5	b=0	j: 1 → 2
L1	0	0	1	0	1	0	1	0	z=4	b=0	j: 2 → 3
L0	0	0	0	0	0	1	0	0	z=7	b=0	j: 3 → 3

✓

$b=1,\ select_1(L_3, 4 - 3) = select_1(L_3, 1) = 1$

$b=0,\ select_0(L_2, 1) = 2$

$b=0,\ select_0(L_1, 2) = 3$

$b=0,\ select_0(L_0, 3) = 3$

Result: j = 4 → **select(1, 1) = 4** ✓ (S[3] = 1, the 2nd occurrence of 1)

Wavelet Matrix vs. Wavelet Tree: A Comparison

Property	Wavelet Tree	Wavelet Matrix	
Memory layout	Pointer-based tree nodes	**Flat bit arrays per level**	**WM wins**
Cache efficiency	Poor (pointer chasing)	**Excellent (sequential access)**	**WM wins**
Implementation	Complex node management	**Simpler — bit-rank only**	**WM wins**
Space (bits)	$n \lceil \log_2 \sigma \rceil$ + o(n) overhead	**$n \lceil \log_2 \sigma \rceil$ + rank & select**	**Same**
access(i)	O(log σ)	**O(log σ)**	**Same**
rank(c, i)	O(log σ)	**O(log σ)**	**Same**
select(c, j)	O(log σ)	**O(log σ)**	**Same**
Practical speedup	—	**2–3× faster in practice**	**WM wins**

Key Takeaways

1. The wavelet matrix stores the same bits as a wavelet tree — but in flat arrays, level by level.
2. A single z-value per level (count of 0s) replaces all the tree pointers needed for navigation.
3. access, rank, and select all run in O(log σ) time using only bit-rank and bit-select primitives.
4. The cache-friendly layout yields 2–3× speedups over pointer-based wavelet trees in practice.
5. Wavelet matrices are foundational in modern compressed genomic indices (FM-index, GBWT, etc.).

"The Wavelet Matrix: An Efficient Wavelet Tree for Large Alphabets"